AN EBOOK FOR ASPIRING ML / AI ENGINEERS

AI / ML PROJECTS EBOOK

12 recruiter-friendly projects, with starter Colab notebooks for every single one.

REGRESSION

CNNs

CREWAI

AUTOGEN

TRANSFORMERS

MLOPS

DSPY

MCP

By Sreemanti

Machine Learning Engineer

2026 Edition

Foreword

When I first got into machine learning I was clueless. I had just discovered the field, got obsessed almost instantly, and started dreaming about building serious ML systems. The problem was, I kept making the same tired beginner projects again and again.

I didn't have anyone to point me in the right direction back then. You do. This ebook is the project list I wish someone had handed me on day one — twelve projects that take you from beginner-level fundamentals to the kind of agentic, multi-modal, production-aware systems that actually move the needle on a resume in 2026.

Every project comes with a starter Jupyter notebook so you can stop reading and start building.

— Sreemanti

How to use this ebook

Each chapter is one project. Read the project page, open the matching starter notebook, and go. The notebooks are designed to be Colab-ready out of the box.

Open a starter notebook in Google Colab

- Go to `https://colab.research.google.com` and sign in with Google.
- Click **File** → **Upload notebook** and select the `.ipynb` from the `notebooks/` folder bundled with this ebook.
- Or simply drag-and-drop the file onto the Colab tab.
- Click **Runtime** → **Change runtime type** and pick GPU when a project needs one (Project 4 and Project 8 will be much faster).
- Run cells top-to-bottom. The first cell of each notebook installs any extra dependencies.

How the projects are rated

Every project is rated on four dimensions on a 1–10 scale:

- **Difficulty** — how hard it is to build end-to-end.
- **Resume value** — how much it helps you stand out to recruiters in 2026.
- **Learning value** — how much you'll actually learn while building it.
- **Real-world impact** — how close it is to systems people use in production.

Ratings are graded relative to each other. A straight-10 is reserved for the projects that stand out across *every* dimension — the ones I'd build first if my goal were a competitive 2026 ML role. Projects with a low Resume score aren't bad choices — they're often the projects that make you a better engineer for everything that comes after.

The 12 projects, at a glance

#	Project	Difficulty	Resume	Notebook
01	Linear Regression From Scratch	3	2	`01_linear_regression_from_scratch.ipynb`
02	Housing Price Predictor	4	3	`02_housing_price_predictor.ipynb`
03	Multi-Agent Job Search (CrewAI)	7	9	`03_multi_agent_job_search_crewai.ipynb`
04	Image Classification (Cats vs Dogs)	6	4	`04_image_classification_cats_dogs.ipynb`
05	AI Travel Planner (A2A Protocol)	7	8	`05_ai_travel_planner_a2a.ipynb`
06	Multi-Agent Research Assistant (AutoGen)	8	8	`06_multi_agent_research_assistant_autogen.ipynb`
07	Recommendation System	7	8	`07_recommendation_system.ipynb`

08	Language Model From Scratch	10	3	`08_language_model_from_scratch.ipynb`
09	Amazon ML Challenge 2024 Solution	9	8	`09_amazon_ml_challenge_2024.ipynb`
10	AI Financial Analyst (DSPy + Flask)	9	9	`10_ai_financial_analyst_dspy_flask.ipynb`
11	MLOps Pipeline	9	10	`11_mlops_pipeline.ipynb`
12	Agentic Dev Workflow (CrewAI + GitHub MCP)	10	10	`12_agentic_dev_workflow_crewai_mcp.ipynb`

PROJECT 01

Linear Regression From Scratch

"If you don't understand how a basic model works, you'll struggle when bigger models start acting weird at 2 a.m."

DIFFICULTY	RESUME VALUE	LEARNING VALUE	REAL-WORLD IMPACT
3/10	2/10	9/10	4/10

What you'll learn

- The math behind linear regression: cost function, gradient descent.
- How weights actually update during training.
- Why feature scaling matters for gradient methods.
- Evaluating regression models with MSE and R^2.

Tech stack

`numpy`, `pandas`, `matplotlib`. **No** scikit-learn for the model itself — that's the point.

Step-by-step plan

1. Generate synthetic `(x, y)` data with a known slope and intercept.

2. Implement the cost function and its analytical gradient by hand.

3. Train using batch gradient descent and watch the loss curve fall.

4. Compare against the closed-form normal equation.

5. Re-run on a real Kaggle regression dataset and write up your findings.

> **Starter notebook:** `notebooks/01_linear_regression_from_scratch.ipynb`
> Open it in Google Colab via **File** → **Upload notebook**, or drag-and-drop the .ipynb onto colab.research.google.com.

Resources

→ Kaggle: Boston Housing dataset

→ Andrew Ng's classic regression notes

PROJECT 02

Housing Price Predictor

"A regression project tons of companies still care about. And yes, the model will probably decide every San Francisco home costs a fortune — that's reality."

DIFFICULTY	RESUME VALUE	LEARNING VALUE	REAL-WORLD IMPACT
4/10	3/10	7/10	6/10

What you'll learn

- Exploratory data analysis on tabular data.
- Handling missing values and categorical features.
- Feature engineering: ratio features, target encoding.
- Comparing linear, tree-based, and gradient-boosted models.

Tech stack

`pandas`, `scikit-learn`, `matplotlib`/`seaborn`, optionally `xgboost`.

Step-by-step plan

1. Load the California Housing or Kaggle 'House Prices' dataset.

2. Run EDA and a correlation heatmap to spot driver features.

3. Train Linear, Ridge, RandomForest, GradientBoosting and compare MAE / R^2.

4. Plot feature importance from the best tree-based model.

5. Wrap the best model in a small FastAPI endpoint as a stretch.

> **Starter notebook:** `notebooks/02_housing_price_predictor.ipynb`
> Open it in Google Colab via **File** → **Upload notebook**, or drag-and-drop the .ipynb onto colab.research.google.com.

Resources

→ Kaggle: House Prices Advanced Regression

→ scikit-learn user guide

PROJECT 03

Multi-Agent Job Search System with CrewAI

"Two birds, one project: it helps with your actual job hunt and proves you can build agentic AI systems people care about."

DIFFICULTY	RESUME VALUE	LEARNING VALUE	REAL-WORLD IMPACT
7/10	9/10	9/10	8/10

What you'll learn

- Designing modular, task-specific agents with CrewAI.
- Wiring agents to LLMs and tools (search, scraping, file I/O).
- Sequential vs hierarchical multi-agent processes.
- Practical agent debugging and prompt iteration.

Tech stack

`crewai`, `langchain`, `openai` (or any LLM provider), `duckduckgo-search`, `python-dotenv`.

Step-by-step plan

1. Define a candidate profile (your real one, ideally).

2. Create three agents: Job Researcher, Resume Tailor, Cover Letter Writer.

3. Define tasks: find roles → tailor resume → draft cover letters.

4. Run a sequential Crew and inspect the per-agent traces.

5. Plug in real search via SerperDev or Tavily; add a SQLite log for application tracking.

> **Starter notebook:** `notebooks/03_multi_agent_job_search_crewai.ipynb`
> Open it in Google Colab via **File** → **Upload notebook**, or drag-and-drop the .ipynb onto colab.research.google.com.

Resources

→ Educative: Build a Job Hunt Agent with CrewAI (free)

→ CrewAI documentation

PROJECT 04

Image Classification (Cats vs Dogs)

"Teach a computer to tell cats from dogs. Sounds easy. It rarely is."

DIFFICULTY	RESUME VALUE	LEARNING VALUE	REAL-WORLD IMPACT
6/10	4/10	8/10	7/10

What you'll learn

- Image preprocessing and augmentation pipelines.
- Building and training a CNN in PyTorch.
- Transfer learning with ResNet on a small dataset.
- Reading a confusion matrix and visualising failures.

Tech stack

`torch`, `torchvision`, `matplotlib`, `scikit-learn`.

Step-by-step plan

1. Grab Oxford-IIIT Pets (built into torchvision) or Kaggle Dogs vs Cats.

2. Build a tfm pipeline: resize, augment, normalise.

3. Transfer-learn from ResNet18 (freeze backbone, replace head).

4. Train, evaluate, and inspect misclassifications.

5. Deploy as a tiny Gradio demo as a stretch goal.

> **Starter notebook:** `notebooks/04_image_classification_cats_dogs.ipynb`
> Open it in Google Colab via **File** → **Upload notebook**, or drag-and-drop the .ipynb onto colab.research.google.com.

Resources

→ Educative: Mastering Image Classification path

→ PyTorch transfer learning tutorial

PROJECT 05

AI Travel Planner with Multi-Agent A2A Protocol

"Multi-agent travel assistant that coordinates flights, hotels and attractions over Google's Agent-to-Agent protocol."

DIFFICULTY	RESUME VALUE	LEARNING VALUE	REAL-WORLD IMPACT
7/10	8/10	8/10	7/10

What you'll learn

- Agent-to-Agent (A2A) protocol fundamentals: cards, messages, capabilities.
- Coordinating specialist agents that own a single domain each.
- Building an orchestrator that ranks itineraries.
- Why structured tool servers beat ad-hoc function calling.

Tech stack

A2A SDK (or hand-rolled JSON envelope), `openai`, `pydantic`, `httpx`.

Step-by-step plan

1. Define agent cards for Flights, Hotels and Attractions.

2. Stub each tool with mock data, then swap in real APIs.

3. Build an Orchestrator that calls each agent and ranks options.

4. Add a budget constraint and have the planner negotiate trade-offs.

5. Wire each agent into Google's official A2A SDK in separate processes.

> **Starter notebook:** `notebooks/05_ai_travel_planner_a2a.ipynb`
> Open it in Google Colab via **File** → **Upload notebook**, or drag-and-drop the .ipynb onto colab.research.google.com.

Resources

→ Google A2A protocol announcement

→ Amadeus travel APIs

PROJECT 06

Multi-Agent Research Assistant with AutoGen

"Stop hand-skimming PDFs. Build agents that find papers, read them, and produce a tidy literature digest."

DIFFICULTY	RESUME VALUE	LEARNING VALUE	REAL-WORLD IMPACT
8/10	8/10	9/10	8/10

What you'll learn

- Multi-agent conversation patterns in AutoGen (group chats, managers).
- Tool-using agents that call arXiv, web search, and a PDF reader.
- Memory and summarisation strategies.
- Producing structured Markdown reports.

Tech stack

`pyautogen`, `arxiv`, `pypdf`, `openai`.

Step-by-step plan

1. Add tools: arXiv search, PDF text extraction.

2. Create three agents: Researcher, Summariser, Critic.

3. Run a GroupChat with a manager, capped at six rounds.

4. Persist findings to a Notion or Obsidian vault.

5. Schedule weekly runs and email yourself the digest.

> **Starter notebook:**
> `notebooks/06_multi_agent_research_assistant_autogen.ipynb`
> Open it in Google Colab via **File** → **Upload notebook**, or drag-and-drop the .ipynb onto colab.research.google.com.

Resources

→ AutoGen documentation

→ arXiv API

PROJECT 07

Recommendation System

"How does Netflix know you'll watch a documentary about competitive cheese rolling? Now you'll know too."

DIFFICULTY	RESUME VALUE	LEARNING VALUE	REAL-WORLD IMPACT
7/10	8/10	9/10	9/10

What you'll learn

- User-item matrix factorisation (SVD).
- Cosine similarity for content-based recommendations.
- Cold-start trade-offs and hybrid recommenders.
- Offline evaluation with HitRate@K and NDCG.

Tech stack

`pandas`, `numpy`, `scipy`, `scikit-learn`. Optional: `surprise`, `implicit`.

Step-by-step plan

1. Download the small MovieLens dataset.

2. Build a sparse user-item matrix and run truncated SVD.

3. Generate top-N recommendations, filtering already-rated items.

4. Build a content-based recommender from genres with TF-IDF + cosine.

5. Hybridise scores and evaluate against a held-out set.

> **Starter notebook:** `notebooks/07_recommendation_system.ipynb`
> Open it in Google Colab via **File** → **Upload notebook**, or drag-and-drop the .ipynb onto colab.research.google.com.

Resources

→ Educative: Collaborative Filtering Recommendation System

→ MovieLens datasets

PROJECT 08

Language Model From Scratch

"Not the next GPT — just enough that those Transformer diagrams stop looking like sacred geometry."

DIFFICULTY	RESUME VALUE	LEARNING VALUE	REAL-WORLD IMPACT
10/10	3/10	10/10	4/10

What you'll learn

- Tokenisation: char-level and byte-pair encoding.
- From n-grams to neural language models.
- Self-attention, multi-head attention, positional encodings.
- Training a tiny GPT on character data and sampling from it.

Tech stack

`torch`, `numpy`, `requests`.

Step-by-step plan

1. Download the Tiny Shakespeare corpus.

2. Build encode/decode and a simple bigram baseline.

3. Implement a single-head attention block from scratch.

4. Stack into a tiny GPT and train for a few hundred steps.

5. Sample and laugh at the early outputs — then keep going.

> **Starter notebook:** `notebooks/08_language_model_from_scratch.ipynb`
> Open it in Google Colab via **File** → **Upload notebook**, or drag-and-drop the .ipynb onto colab.research.google.com.

Resources

→ Karpathy: Let's build GPT from scratch (YouTube)

→ The Annotated Transformer

PROJECT 09

Amazon ML Challenge 2024 Solution

"Extract entity values (weight, voltage, dimensions) directly from product images. RAG didn't cut it — fine-tuning did."

DIFFICULTY	RESUME VALUE	LEARNING VALUE	REAL-WORLD IMPACT
9/10	8/10	10/10	9/10

What you'll learn

- Vision-language pipelines.
- When RAG is the wrong tool and fine-tuning wins.
- LoRA / PEFT-style fine-tuning of multimodal models.
- Building structured-output extractors with a JSON schema.

Tech stack

`transformers`, `peft`, `accelerate`, `bitsandbytes`, `Pillow`, `pytesseract` (baseline).

Step-by-step plan

1. Build an OCR + regex baseline. Be ready to be embarrassed by it.

2. Run zero-shot inference with a small VLM (Qwen2-VL-2B or Phi-3-Vision).

3. Curate a small labelled set and LoRA fine-tune.

4. Compare baseline vs zero-shot vs fine-tuned on a fixed eval split.

5. Submit to Amazon ML Challenge 2025.

> **Starter notebook:** `notebooks/09_amazon_ml_challenge_2024.ipynb`
> Open it in Google Colab via **File** → **Upload notebook**, or drag-and-drop the .ipynb onto colab.research.google.com.

Resources

→ Hugging Face PEFT documentation

→ Qwen2-VL model card

PROJECT 10

AI Financial Analyst with DSPy and Flask

"Pull live market data, compute indicators, and let a DSPy program produce a written analysis behind a Flask endpoint."

DIFFICULTY	RESUME VALUE	LEARNING VALUE	REAL-WORLD IMPACT
9/10	9/10	9/10	9/10

What you'll learn

- DSPy program composition: Signatures, Modules, Optimizers.
- Pulling and cleaning financial time-series data.
- Computing classic indicators: returns, volatility, SMAs, RSI.
- Wrapping a model in a small Flask service.

Tech stack

`dspy`, `yfinance`, `pandas`, `flask`.

Step-by-step plan

1. Fetch one year of daily prices via `yfinance`.

2. Compute returns, annualised volatility, and 50/200-day SMAs.

3. Define a DSPy Signature for ticker -> analysis + buy/hold/sell call.

4. Expose the program through a `/analyse` Flask endpoint.

5. Add NewsAPI headlines and tune with `MIPROv2`.

> **Starter notebook:** `notebooks/10_ai_financial_analyst_dspy_flask.ipynb`
> Open it in Google Colab via **File** → **Upload notebook**, or drag-and-drop the .ipynb onto colab.research.google.com.

Resources

→ DSPy documentation

→ yfinance on PyPI

PROJECT 11

MLOps Pipeline

"The side of ML companies actually pay for: versioning, monitoring, retraining."

DIFFICULTY	RESUME VALUE	LEARNING VALUE	REAL-WORLD IMPACT
9/10	10/10	10/10	10/10

What you'll learn

- Experiment tracking with MLflow.
- Data and model versioning with DVC.
- Drift monitoring with Evidently.
- Automated retraining triggers and a FastAPI inference service.

Tech stack

`mlflow`, `dvc`, `evidently`, `scikit-learn`, `fastapi`, `uvicorn`.

Step-by-step plan

1. Train a baseline RandomForest and log params/metrics/artefact to MLflow.

2. Generate a 'production-like' shifted dataset and run an Evidently drift report.

3. Package an inference service with FastAPI loading the MLflow run.

4. Define a retraining trigger: drift > 0.5 OR rolling MAE > threshold.

5. Promote a new model from `staging` to `production` aliases.

> **Starter notebook:** `notebooks/11_mlops_pipeline.ipynb`
> Open it in Google Colab via **File** → **Upload notebook**, or drag-and-drop the .ipynb onto colab.research.google.com.

Resources

→ Educative: Building a Machine Learning Pipeline from Scratch

→ MLflow documentation

PROJECT 12

Agentic Dev Workflow with CrewAI + GitHub MCP

"Turn a repo's issues, PRs and branches into a clean, human-readable digest using CrewAI agents talking to a GitHub MCP server."

DIFFICULTY	RESUME VALUE	LEARNING VALUE	REAL-WORLD IMPACT
10/10	10/10	10/10	10/10

What you'll learn

- Connecting agents to MCP (Model Context Protocol) servers.
- Coordinating CrewAI agents around real engineering data.
- Producing weekly engineering digests automatically.
- The 'agentic ops' pattern: agents + structured tool servers.

Tech stack

`crewai`, `mcp` (Python client), GitHub MCP server, `openai`.

Step-by-step plan

1. Stand up the github-mcp-server as a subprocess, talking JSON-RPC over stdio.

2. Wrap the MCP tools (issues, PRs, branches) as plain Python callables.

3. Define two CrewAI agents: Repo Scout and Engineering Editor.

4. Run sequentially to produce a 300-word Friday digest.

5. Schedule it weekly and post the digest to Slack via webhook.

> **Starter notebook:** `notebooks/12_agentic_dev_workflow_crewai_mcp.ipynb`
> Open it in Google Colab via **File** → **Upload notebook**, or drag-and-drop the .ipynb onto colab.research.google.com.

Resources

→ github-mcp-server (official)

→ Model Context Protocol spec

Where to go from here

If you build any one of these well, you'll have learned more than from a dozen tutorials watched passively. If you build several, you'll have a portfolio that opens conversations.

If you're picking just one to start

Start with **Project 3 — Multi-Agent Job Search with CrewAI**. It's the rare project that pays you back twice: you learn a current, in-demand stack, and the system itself helps you find the next role. The Educative project for it is free to follow:

Build a Job Hunt Agent Workflow with CrewAI and Python →

Stuck? Reach out

If you get stuck while building any of these, drop your question on YouTube, connect on LinkedIn, or book a 1:1 career counselling session on Topmate. Confusion is part of the process. Don't get stuck silently.

— Sreemanti, 2026 Edition